

LH Nautical

Desafio de Ciência e Engenharia de Dados

Relatório Técnico Consolidado — Questões 1 a 7 + Explorações Adicionais

De dados brutos em CSV a um dashboard de insights de negócio

Autor: Rafael Silva Lemos Ferreira

Escopo: Modelagem, carga, EDA, SQL analítico e modelos de ML (forecast e recomendação)

Stack: PostgreSQL · Python (psycopg2, pandas, scikit-learn) · Power BI

Material complementar ao dashboard (Dashboard_Indicium.pbix/.pdf) entregue em anexo.

Sumário Executivo

Este relatório consolida as sete etapas do desafio de dados da LH Nautical: da análise exploratória inicial (respondendo à pergunta do Sr. Almir — “posso confiar nesses dados?”) até a construção do schema, carga no PostgreSQL, análises de negócio em SQL e dois modelos de machine learning (previsão de demanda e sistema de recomendação). O material complementa o dashboard em Power BI (anexado separadamente em .pbix/.pdf), trazendo o raciocínio técnico, o código utilizado e as interpretações de negócio por trás de cada visual.

Principais achados:

- A tabela **orders** (48.998 pedidos, período de 01/01/2020 a 31/12/2026) é utilizável para análises, mas exige atenção: 49,25% dos registros não têm vendedor atribuído (**salesperson_id** nulo).
- Os clientes de maior ticket médio e diversidade de categorias (≥ 13 categorias distintas) concentram 11% do volume de compras na categoria **Hélices**.
- Não há um dia da semana com sazonalidade forte de vendas — a variação entre o melhor e o pior dia é de apenas ~1,5 ponto percentual sobre o total.
- O modelo baseline de média móvel (3 meses) para previsão de demanda apresentou MAE de 19,44 unidades no 1º trimestre de 2026, reagindo com atraso a uma tendência de queda real na demanda.
- O sistema de recomendação por similaridade de cosseno identificou produtos complementares/afins ao 'Motor de Popa 1949' com base no comportamento de coocorrência de compra entre clientes.
- Nenhum dos 10 clientes de maior **lucro** acumulado aparece entre os 10 de maior **faturamento** (Questão 4) — margem de produto pesa mais que volume de compra.
- Os produtos com maior prejuízo por devolução (Bússola de Bordo 8282 e Tinta Antifouling 6853, ~R\$ 58 mil cada) concentram quase metade do valor total devolvido pelo Top 10.

Estrutura do documento: cada seção a seguir corresponde a uma questão do desafio, trazendo o cenário, o código desenvolvido, os resultados obtidos e a interpretação de negócio.

QUESTÃO 1 Análise Exploratória (EDA) — tabela orders

Cenário

Antes de qualquer modelagem, era preciso responder de forma objetiva se os dados de **orders** são confiáveis para embasar decisões. A análise foi feita exclusivamente em SQL, sem qualquer limpeza ou tratamento — apenas observação, agregação e descrição dos dados como se apresentam.

Código (SQL)

```
-- Visão geral: linhas, completude por coluna e intervalo de datas
SELECT
COUNT(*) AS total_linhas,
COUNT(salesperson_id) AS salesperson_id_preenchidos,
COUNT(id) AS id_preenchidos,
COUNT(status) AS status_preenchidos
FROM orders;

SELECT MIN(created_at) AS data_minima, MAX(created_at) AS data_maxima
FROM orders;

-- Estatísticas numéricas da coluna total (outliers)
SELECT
MIN(total) AS valor_minimo,
MAX(total) AS valor_maximo,
ROUND(AVG(total), 2) AS valor_medio,
ROUND(STDDEV(total), 2) AS desvio_padrao,
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY total) AS mediana,
PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY total) AS p95,
PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY total) AS p99
FROM orders;

-- Verificação de inconsistências (valores <= 0)
SELECT COUNT(*) AS totais_invalidos FROM orders WHERE total <= 0;
```

Arquivo completo: *EDA.sql* (anexo)

Resultados

Total de pedidos	Período analisado	Nulos em salesperson_id	% nulos
48.998	01/01/2020 a 31/12/2026	24.131	49,25%

Mínimo	Mediana (P50)	Média	P95	P99	Máximo
R\$ 32,62	R\$ 25.917,84	R\$ 28.704,99	R\$ 64.869,03	R\$ 81.764,23	R\$ 127.262,02

Diagnóstico (Interpretação)

Com base na análise exploratória, a tabela **orders** é confiável como ponto de partida, mas não está pronta para decisões de negócio sem tratamento prévio. Em **total**, o valor máximo (R\$ 127.262,02) frente à média (R\$ 28.704,99) e mediana (R\$ 25.917,84) indica possíveis outliers; porém, a progressão gradual até o P99 (R\$ 81.764,23) sugere que são vendas reais de alto valor (atacado/B2B), e não erros — ainda assim, distorcem a média e pedem segmentação. Já **salesperson_id** apresenta 24.131 valores nulos (~49,25% dos registros), o que compromete análises de performance por vendedor. A tabela permite análises exploratórias

gerais, mas exige tratamento dos nulos/outliers e relacionamento com outras tabelas (ex: customers, salespeople) para sustentar decisões de negócio mais específicas.

QUESTÃO 2 Construção Automática do Schema

Cenário

O ERP de origem só disponibiliza os dados em arquivos CSV, sem acesso direto ao banco. Foi desenvolvido um script em Python 3 puro (sem pandas ou bibliotecas externas de dados) que lê todos os CSVs de um diretório, infere o tipo de cada coluna e gera um único arquivo **schema.sql** com o *CREATE TABLE* de cada tabela, pronto para o PostgreSQL.

Lógica principal

```
# Colunas identificadoras (CPF, CNPJ, telefone, CEP...) são
# sempre forçadas para VARCHAR, mesmo parecendo numéricas,
# pois nunca fazemos conta com elas.
COLUNAS_FORCAR_VARCHAR = [
    "cpf", "cnpj", "tax_id", "phone", "telefone",
    "barcode_ean", "nfe_access_key", "cep", "zip_code", "ncm_code"
]

# Para cada coluna, tenta converter cada valor na ordem:
# INTEGER -> NUMERIC -> BOOLEAN -> DATE -> TIMESTAMP -> VARCHAR
# (fallback seguro em caso de tipo misto)
```

Resultado

O script percorreu recursivamente o diretório de CSVs e gerou **24 tabelas** no arquivo **schema.sql** (addresses, customers, orders, order_items, payments, products, categories, stock_movements, entre outras), cobrindo todo o modelo de dados do ERP da LH Nautical.

Trecho gerado (exemplo)

```
CREATE TABLE orders (
  id INTEGER,
  order_number VARCHAR,
  channel VARCHAR,
  customer_id INTEGER,
  salesperson_id INTEGER,
  location_id INTEGER,
  status VARCHAR,
  subtotal NUMERIC,
  discount_amount NUMERIC,
  total NUMERIC,
  placed_at TIMESTAMP,
  created_at TIMESTAMP,
  updated_at TIMESTAMP
);
```

Arquivos completos: *Schema.py* e *schema.sql* (anexo)

QUESTÃO 3 Carregamento dos Dados

Cenário

Com o schema definido, o próximo passo foi carregar todos os CSVs no banco PostgreSQL, respeitando as premissas de não remover nulos nem corrigir caracteres especiais. O script utiliza **psycopg2** com **COPY ... FROM STDIN** (muito mais rápido que INSERT linha a linha) e foi desenhado para ser **idempotente**: antes de carregar cada tabela, verifica se ela já possui dados, evitando duplicação caso o script seja executado mais de uma vez.

Código (trecho relevante)

```
# Verificação de idempotência antes de cada carga
cursor.execute(sql.SQL("SELECT COUNT(*) FROM {}".format(sql.Identifier(nome_tabela)))
quantidade = cursor.fetchone()[0]

if quantidade > 0:
    print(f"Tabela '{nome_tabela}' já possui {quantidade} linhas. Não será carregada novamente.")
    continue

comando_copy = sql.SQL(
    "COPY {} FROM STDIN WITH (FORMAT csv, HEADER true)"
).format(sql.Identifier(nome_tabela))
cursor.copy_expert(comando_copy, arquivo_csv)
```

Arquivo completo: *carregamento.py* (anexo)

Questão 3.2 — Validação

Total de linhas somadas das tabelas customers, orders, order_items e payments:

customers	orders	order_items	payments	TOTAL
2.000	48.998	147.320	53.546	251.864

Resposta: 251.864 linhas. Os números são consistentes com o esperado: order_items tem quase 3x mais linhas que orders (média de ~3 itens por pedido), e payments é ligeiramente maior que orders, compatível com pedidos pagos em mais de uma transação/parcela.

QUESTÃO 4 Análise de Clientes de Elite

Cenário

A diretoria da LH Nautical queria identificar os **clientes fiéis**: não quem compra muito uma única vez, mas quem tem gasto médio alto por transação e navega por diversas categorias da loja. O objetivo final era mapear o que esses clientes de elite consomem, para replicar o comportamento em outros segmentos.

Código (SQL)

```
WITH clientes_orders AS (  
  SELECT customer_id, SUM(total) AS faturamento_total,  
         COUNT(id) AS frequencia,  
         SUM(total)/COUNT(id) AS ticket_medio  
  FROM orders  
  GROUP BY customer_id  
,  
clientes_products AS (  
  SELECT o.customer_id, COUNT(DISTINCT pr.category_id) AS diversidade_categorias  
  FROM orders o  
  JOIN order_items oi ON o.id = oi.order_id  
  JOIN product_variants pv ON oi.product_variant_id = pv.id  
  JOIN products pr ON pv.product_id = pr.id  
  GROUP BY o.customer_id  
,  
clientes_elite AS (  
  SELECT co.customer_id, co.faturamento_total, co.frequencia,  
         co.ticket_medio, cp.diversidade_categorias  
  FROM clientes_orders co  
  JOIN clientes_products cp ON co.customer_id = cp.customer_id  
  WHERE cp.diversidade_categorias >= 13  
  ORDER BY co.ticket_medio DESC, co.customer_id ASC  
  LIMIT 10  
)  
SELECT c.id AS category_id, c.name AS category_name,  
       SUM(oi.quantity) AS quantidade_total  
FROM clientes_elite ce  
JOIN orders o ON ce.customer_id = o.customer_id  
JOIN order_items oi ON o.id = oi.order_id  
JOIN product_variants pv ON oi.product_variant_id = pv.id  
JOIN products p ON pv.product_id = p.id  
JOIN categories c ON p.category_id = c.id  
GROUP BY c.id, c.name  
ORDER BY quantidade_total DESC  
LIMIT 1;
```

Arquivo completo: *analise_cliente.sql* (anexo)

Resultado

Os 10 clientes de elite (ticket médio mais alto, com 14 categorias distintas compradas cada) têm a categoria **Hélices** como líder de volume, com 492 unidades — representando 11% de tudo o que esse grupo comprou. As categorias seguintes são Coletes Salva-Vidas (393), Eletrônica Náutica (392) e Âncoras (387).

Questão 4.2 — Explicação

Mapeamento da cadeia de chaves:

orders não se liga direto a categories, então a cadeia percorrida foi: *orders* → *order_items* → *product_variants* → *products* → *categories*. Cada item de pedido leva a uma variação de produto, que leva a um produto, que carrega o *category_id*. Com isso, agrupei por categoria e somei *oi.quantity*, restringindo

apenas aos pedidos dos 10 clientes elite.

Lógica de filtro de diversidade mínima:

Numa CTE separada, contei `COUNT(DISTINCT pr.category_id)` por cliente, percorrendo a mesma cadeia até `category_id`. Isso dá o número de categorias diferentes que cada cliente comprou. Depois apliquei `WHERE diversidade_categorias >= 13` para manter só quem atende ao critério de elite.

Garantia de refletir apenas os Top 10:

O JOIN da query final parte da CTE `clientes_elite` (já filtrada e limitada aos 10 via `ORDER BY ticket_medio DESC, customer_id ASC LIMIT 10`), em vez de partir de `orders` inteira. Assim, só pedidos dos 10 clientes selecionados entram na soma de `quantity` — os demais nunca são considerados.

QUESTÃO 5 Sazonalidade Semanal de Vendas

Cenário

O objetivo foi calcular a média de vendas por dia da semana no canal **POS** (loja física), garantindo que dias sem nenhuma venda registrada não fossem ignorados no cálculo — o que exigiu a construção de uma **dimensão de calendário** cobrindo todo o período analisado.

Código (SQL)

```
WITH periodo AS (
  SELECT MIN(placed_at::DATE) AS data_inicio, MAX(placed_at::DATE) AS data_fim
  FROM orders WHERE channel = 'pos'
),
calendario AS (
  SELECT data::DATE AS data,
  EXTRACT(ISODOW FROM data)::INT AS numero_dia,
  CASE EXTRACT(ISODOW FROM data)
  WHEN 1 THEN 'Segunda-feira' WHEN 2 THEN 'Terça-feira'
  WHEN 3 THEN 'Quarta-feira' WHEN 4 THEN 'Quinta-feira'
  WHEN 5 THEN 'Sexta-feira' WHEN 6 THEN 'Sábado'
  WHEN 7 THEN 'Domingo'
  END AS semanal
  FROM periodo, generate_series(data_inicio, data_fim, INTERVAL '1 day') AS data
),
vendas_diarias AS (
  SELECT c.data, c.numero_dia, c.semanal,
  COALESCE(SUM(o.total), 0) AS vendas
  FROM calendario c
  LEFT JOIN orders o
  ON c.data = o.placed_at::DATE AND o.channel = 'pos'
  GROUP BY c.data, c.numero_dia, c.semanal
)
SELECT numero_dia, semanal, ROUND(AVG(vendas), 2) AS media_vendas
FROM vendas_diarias
GROUP BY numero_dia, semanal
ORDER BY media_vendas;
```

Arquivo completo: *dimensao_calendario.sql* (anexo)

Resultado

Seg	Ter	Qua	Qui	Sex	Sáb	Dom
158,24 Mil	166,12 Mil	173,61 Mil	157,15 Mil	170,19 Mil	164,86 Mil	157,62 Mil

Não há um dia dominante: a variação entre o melhor (Quarta, 15,13% do total semanal) e o pior (Quinta, 13,69%) fica em torno de 1,5 ponto percentual — uma distribuição bastante equilibrada ao longo da semana, sem sazonalidade forte por dia.

Questão 5.2 — Explicação

Por que usar uma tabela de calendário em vez de agrupar direto a tabela de vendas?

Agrupando direto de orders, dias sem nenhuma venda simplesmente não aparecem — não existe linha pra eles. Isso infla a média, pois ela seria calculada só sobre os dias "bons". A tabela de calendário garante uma linha para cada data do período, e o LEFT JOIN com COALESCE(..., 0) preenche os dias sem venda com zero, em vez de ignorá-los.

O que aconteceria se um dia da semana tivesse muitos dias sem venda?

Sem o calendário, esses dias zerados seriam excluídos do cálculo, inflando artificialmente a média daquele dia — dando a impressão de que ele vende bem quando na verdade é fraco na maior parte do tempo. Com o calendário completo, cada dia sem venda entra como zero, puxando a média pra baixo e refletindo a realidade.

QUESTÃO 6 Previsão de Demanda

Cenário

Foi construído um baseline de previsão de demanda mensal para o produto "Bússola de Bordo 702", usando **média móvel de 3 meses**, com avaliação no 1º trimestre de 2026 via MAE (Mean Absolute Error).

Código (trechos relevantes)

```
# Série mensal completa, preenchendo meses sem venda com 0
periodo_completo = pd.period_range(start=df_mensal.index.min(), end="2026-03", freq="M")
df_mensal = df_mensal.reindex(periodo_completo, fill_value=0).reset_index()

treino = df_mensal[df_mensal["mes"] <= "2025-12"].copy()
teste = df_mensal[(df_mensal["mes"] >= "2026-01") & (df_mensal["mes"] <= "2026-03")].copy()

# Previsão: média dos 3 meses anteriores (sempre dados reais, nunca previstos)
for i in range(len(teste)):
    indice = inicio_teste + i
    ultimos_3 = dados["quantity"].iloc[indice - 3:indice]
    previsoes.append(ultimos_3.mean())

mae = mean_absolute_error(resultado["quantity"], resultado["previsao"])
```

Arquivo completo: *previsao_demanda.py* (anexo)

Resultado

Mês	Real (unid.)	Previsto (unid.)
Jan/2026	79	39
Fev/2026	68	54
Mar/2026	60	56

MAE: 19,44 unidades. O real caiu de forma consistente ao longo do trimestre, enquanto a previsão reagiu com atraso e subestimou a demanda de janeiro (39 vs. 79 reais).

Questão 6.3 — Explicação

Como o baseline foi construído?

Para cada mês do 1º trimestre de 2026, a previsão é a média das quantidades vendidas nos 3 meses imediatamente anteriores (dados reais, não previstos). A série mensal foi completada com `reindex + fill_value=0`, garantindo que meses sem venda entrem como zero na média.

Como evitou data leakage?

Cada previsão usa apenas os 3 meses anteriores ao mês sendo previsto — nunca dados do próprio mês ou de meses futuros. Com a janela deslizante (`indice-3:indice`), o modelo nunca "enxerga" o valor real do mês que está tentando prever antes de gerar a previsão. Treino (até dez/2025) e teste (jan–mar/2026) são separados por data, sem embaralhar a ordem temporal.

Uma limitação do modelo:

A média móvel simples assume que o padrão recente se mantém estável, mas não captura tendência nem sazonalidade. Como a demanda estava em queda consistente, o modelo reagiu com atraso — visível no

MAE de ~19 unidades: o baseline serve como referência simples, mas não é ideal para produtos com padrão de venda mais dinâmico.

QUESTÃO 7 Sistema de Recomendação de Produtos

Cenário

Foi construído um sistema de recomendação item-a-item baseado em **similaridade de cosseno**, identificando os 5 produtos mais similares ao "Motor de Popa 1949" com base no comportamento de compra conjunta dos clientes (filtragem colaborativa).

Código (trechos relevantes)

```
df_compras = (
    order_items
    .merge(orders[["id", "customer_id"]], left_on="order_id", right_on="id")
    .merge(product_variants[["id", "product_id"]], left_on="product_variant_id", right_on="id")
    [["customer_id", "product_id"]]
    .drop_duplicates()
)

matriz = pd.crosstab(df_compras["customer_id"], df_compras["product_id"])
similaridade = cosine_similarity(matriz.T) # transposta: similaridade entre produtos

ranking = (
    df_similaridade.loc[produto_id]
    .drop(index=produto_id)
    .sort_values(ascending=False)
    .head(5)
)
```

Arquivo completo: `sistema_recomendacao.py` (anexo)

Resultado — Top 5 recomendações para "Motor de Popa 1949"

#	Produto	Similaridade
1	Motor de Popa 5331	0,2566
2	Cabo Náutico 2105	0,2562
3	Vela Mestra 1913	0,2558
4	Cabo Náutico 9048	0,2393
5	GPS Plotter 6249	0,2377

Questão 7.3 — Explicação

Como a matriz foi construída?

Cruzei `order_items` com `orders` (para pegar o `customer_id`) e com `product_variants` (para pegar o `product_id`, já que o item referencia uma variação, não o produto direto), montando pares únicos (`customer_id`, `product_id`) com `drop_duplicates()`. Com `pd.crosstab`, transformei isso numa matriz cliente x produto, transposta para calcular similaridade entre produtos.

O que significa a similaridade de cosseno nesse contexto?

Mede o quão parecido é o padrão de coocorrência de clientes entre dois produtos — o quanto os mesmos clientes tendem a comprar ambos. Quanto mais próximo de 1, maior a sobreposição de público; quanto mais próximo de 0, menos clientes em comum. Não mede semelhança de características do produto, só o

comportamento agregado de compra.

Uma limitação do método:

Sofre com **cold start**: produtos novos ou com poucas vendas têm vetores esparsos, tornando a similaridade pouco confiável. Também ignora frequência e contexto da compra, e não incorpora informação de conteúdo (categoria, marca, preço) — apenas o comportamento agregado.

EXTRA

Explorações Adicionais — Prejuízo e Lucro

Cenário

Complementando as análises obrigatórias, foram desenvolvidas duas explorações adicionais sugeridas para o material de apoio do dashboard: um **ranking de prejuízo por produto** (baseado em devoluções) e um **ranking de lucro acumulado por cliente** (faturamento menos custo, diferente do ranking de faturamento bruto já calculado na Questão 4).

Prejuízo por Produto — Código (SQL)

```
SELECT
p.id AS product_id,
p.name AS product_name,
COUNT(ri.id) AS qtd_devolucoes,
SUM(ri.unit_refund_amount * ri.quantity) AS prejuizo_total
FROM return_items ri
JOIN returns r
ON ri.return_id = r.id
JOIN order_items oi
ON ri.order_item_id = oi.id
JOIN product_variants pv
ON oi.product_variant_id = pv.id
JOIN products p
ON pv.product_id = p.id
GROUP BY p.id, p.name
ORDER BY prejuizo_total DESC
LIMIT 10
```

Resultado

Prejuízo total (Top 10)	Qtd. de devoluções (Top 10)
R\$ 540,30 mil	57

Produto	Prejuízo (R\$)
Bússola de Bordo 8282	58 mil
Tinta Antifouling 6853	58 mil
Motor de Popa 6404	55 mil
Âncora Bruce 7665	55 mil
Bateria Náutica 1945	54 mil

Os cinco produtos com maior prejuízo somam praticamente metade do valor total das devoluções do Top 10, com destaque para **Bússola de Bordo 8282** e **Tinta Antifouling 6853** (R\$ 58 mil cada). Recomenda-se investigar se há um padrão comum entre esses produtos (defeito de fabricação, descrição enganosa, logística) que explique a concentração de devoluções.

Lucro Acumulado por Cliente — Código (SQL)

```
WITH margem_itens AS (  
  SELECT  
    orders.customer_id,  
    order_items.quantity * (order_items.unit_price - product_variants.cost_price) AS lucro_item  
  FROM orders  
  JOIN order_items ON order_items.order_id = orders.id  
  JOIN product_variants ON product_variants.id = order_items.product_variant_id  
)  
SELECT  
  customer_id,  
  ROUND(SUM(lucro_item), 2) AS lucro_acumulado  
FROM margem_itens  
GROUP BY customer_id  
ORDER BY lucro_acumulado DESC  
LIMIT 10
```

Resultado

customer_id	1434	643	1563	177	740
Lucro acumulado	R\$ 0,60 Mi	R\$ 0,60 Mi	R\$ 0,57 Mi	R\$ 0,56 Mi	R\$ 0,56 Mi

Insight — Faturamento vs. Lucro

Comparando este ranking com o de **faturamento** obtido na Questão 4 (clientes 22, 1477, 929, 1116, 1691, 774, 1470, 1599, 965 e 1722), **nenhum cliente aparece nos dois rankings**. Isso confirma que quem mais fatura para a LH Nautical não é necessariamente quem mais gera lucro: clientes de alto faturamento podem estar concentrados em produtos de margem baixa, enquanto os clientes de maior lucro compram um mix de produtos com margem mais favorável, mesmo com faturamento total menor. Esse achado sugere que a área comercial deveria monitorar **margem**, e não apenas volume de vendas, ao priorizar o relacionamento com clientes estratégicos.

Considerações Finais e Anexos

Este relatório, junto ao dashboard em Power BI (Dashboard_Indicium), demonstra o fluxo completo de um projeto de dados: da ingestão bruta em CSV, passando por modelagem de schema, carga idempotente em PostgreSQL, análise exploratória com diagnóstico de qualidade, até análises de negócio em SQL e dois modelos de machine learning aplicados a problemas reais de negócio (previsão de demanda e recomendação de produtos).

Arquivos anexados a esta entrega:

- **Dashboard_Indicium.pbix / .pdf** — dashboard interativo com 7 páginas (Qualidade dos Dados, Clientes de Alto Valor, Lucro por Cliente, Análise Semanal, Prejuízo por Produto, Previsão de Demanda, Similaridade de Produtos).
- **EDA.sql** — código completo da Questão 1.
- **Schema.py / schema.sql** — código e resultado da Questão 2.
- **carregamento.py** — código completo da Questão 3.
- **analise_cliente.sql** — código completo da Questão 4.
- **dimensao_calendario.sql** — código completo da Questão 5.
- **previsao_demanda.py** — código completo da Questão 6.
- **sistema_recomendacao.py** — código completo da Questão 7.